A

MINI PROJECT-1B REPORT

On

"A SHORT-RANGE SYSTEM USING ARDUINO UNO"

BY

Aamir Kazi	122A2001
Aayush Pandey	122A2002
Sahil Gurkhe	122A2043

UNDER THE GUIDANCE OF

Prof. Shyamala Mathi

SUBMITTED IN PARTIAL FULFILLMENT FOR THE DEGREE OF

SECOND YEAR OF ENGINEERING

IN

ELECTRONICS AND TELECOMMUNICATION

DEPARTMENT OF ELECTRONICS AND TELECOMMUNICATION

SIES GRADUATE SCHOOL OF TECHNOLOGY

NERUL, NAVI MUMBAI – 400706

ACADEMIC YEAR

2023- 2024

CERTIFICATE

This is to certify that the project titled “” carried out by following students of S.E in Electronics and Telecommunication:

Aamir Kazi	122A2001
Aayush Pandey	122A2002
Sahil Gurkhe	12A2043

The project report is submitted in partial fulfilment of second year of engineering Electronics and Telecommunication from University of Mumbai during the academic year 2023 - 2024

Prof. Shyamala Mathi	Dr.PreetiHemnani
Internal Guide	**Head Of Department**

We have examined this report as per University requirements at SIES Graduate School of Technology, Nerul (E), Navi Mumbai on _____

Name of External Examiner : _____

Signature with Date : _____

Name of Internal Examiner : _____

Signature with Date :_____

ABSTRACT

RADAR is a detection system that uses radio waves to determine the characteristics of the detected objects such as: range, height, direction, or the speed of objects. In this paper, we designed a radar system that uses an ultrasonic sensor to detect objects. In this paper, the ultrasonic is used to measure the distance between the radar and any object-based non-contact technology. The project begins with a comprehensive overview of radar technology, elucidating the underlying principles of electromagnetic wave propagation, reflection, and Doppler effect. It navigates through the intricacies of radar signal processing, emphasizing the significance of pulse modulation techniques, waveform design, and digital signal processing algorithms for accurate target detection and tracking. Whereas the movement of the sensor is controlled by using a small servo motor. This radar controlled using the Arduino Uno board as a microcontroller. The signal received from the sensor would be processed using "Processing Development Environment Software" then the result would be shown on a PC screen.

Keywords: Radar, Ultrasonic Sensor, Arduino Uno, Servo motor.

ACKNOWLEDGEMENT

It gives us immense pleasure to thank Dr. K. Lakshmisudha, our principal for extending her support to carry out this project.

We also thank to our HOD Dr. Preeti Hemnani We wish to express our deep appreciation to our Internal Guide, Prof. Pranavi Mhatre for her guidance, help and insightful suggestions, which helped us in completing the planning and analysis phase of our project in time.

Also, we would like to thank the entire faculty of EXTC department for their valuable ideas and timely assistance in this project. Finally, we would like to thank our teaching and non-teaching staff members of our college for their support, in facilitating timely completion of this project.

TABLE OF CONTENT

	Abstract	i
	Acknowledgement	ii
	Table of content	iii
	List of Figure	iv
1	Introduction	1
2	Working Principle	2
3	Circuit Description	3-4
	3.1. Block diagram	
	3.2. Circuit diagram	
4	Component Description	6-10
5	Software Description	10-15
6	Advantages	16
7	Disadvantages	17
8	Scope	18
9	Conclusion	19
10	References	20

LIST OF FIGURE

(Figure 1.1) IMPLEMENTATION OF CIRCUIT ON BREADBOARD 3

(Figure 1.2) BLOCK DIAGRAM OF THE SYSTEM 3

(Figure 1.3) ACTUAL IMAGE OF THE PROJECT 4

(Figure 1.4) ARDUINO UNO PIN OUT 5

(Figure 1.5) ICSP HEADER PIN OUT 6

(Figure 1.6) HC-S04 ULTRASONIC SENSOR 7

(Figure 1.7) HC-S04 ULTRASONIC SENSOR 9

INTRODUCTION

In an era marked by technological advancement and innovation, the evolution of sensing systems has played a pivotal role in shaping modern societies. Among these sensing technologies, radar stands out as a cornerstone of detection, tracking, and situational awareness across diverse domains. While radar systems have traditionally been associated with long-range surveillance and aerospace applications, recent years have witnessed a paradigm shift towards the development and adoption of short-range radar detection systems. These systems, characterized by their compact form factor, high resolution, and versatility, have revolutionized various industries, from automotive safety to industrial automation and security.

The journey of short-range radar detection systems traces back to the early 20th century, with the pioneering work of scientists such as Christian Hülsmeyer and Sir Robert Watson-Watt, who laid the foundation for modern radar technology. Initially conceived for military purposes, radar systems were primarily utilized for long-range detection of aircraft and ships during World War II. However, advancements in semiconductor technology, signal processing algorithms, and miniaturization techniques have democratized radar technology, enabling its integration into everyday devices and applications.

Signal processing lies at the heart of radar technology, facilitating the extraction of useful information from received radar signals amidst noise, clutter, and interference. Digital signal processing (DSP) algorithms, including matched filtering, pulse compression, and adaptive beamforming, are employed to enhance target detection, mitigate clutter, and improve signal-to-noise ratio (SNR). Time-domain and frequency-domain processing techniques enable range and Doppler estimation, enabling the characterization of target properties such as distance, velocity, and size.

WORKING PRINCIPLE

The working principle of a short-range radar detection system is grounded in the fundamental principles of radar technology, which relies on the transmission, propagation, and reception of electromagnetic waves to detect the presence and characteristics of objects in the surrounding environment.

Transmitting Electromagnetic Waves: The radar system begins by emitting electromagnetic waves, typically in the microwave or millimetre-wave frequency bands. These waves are generated by the radar transmitter and propagate outward into the surrounding space.

Wave Propagation and Reflection: As the electromagnetic waves propagate through the environment, they encounter various objects in their path. When the waves encounter an object, a portion of the transmitted energy is reflected towards the radar system due to the phenomenon of reflection.

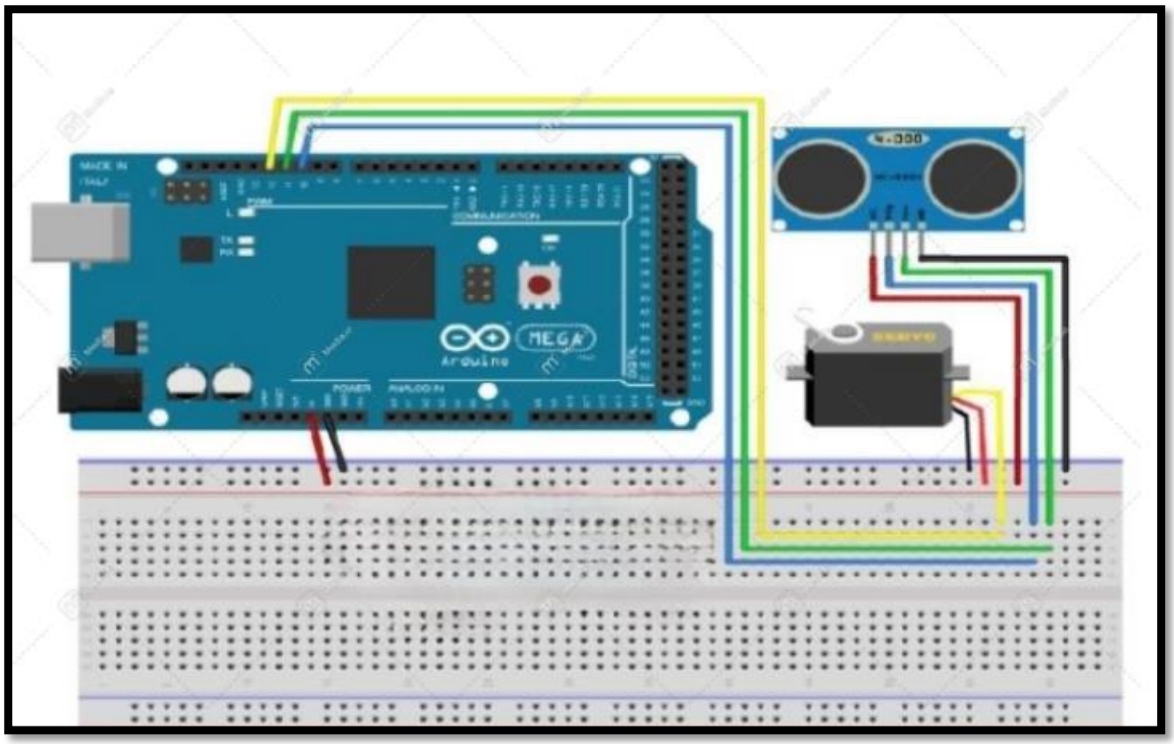
Receiving Reflected Signals: The receiver is typically equipped with a sensitive antenna that captures the incoming signals.

Signal Processing: Once the reflected signals are received, they undergo signal processing to extract relevant information about the detected objects. This processing involves analysing the characteristics of the received signals, including amplitude, phase, frequency, and time delay.

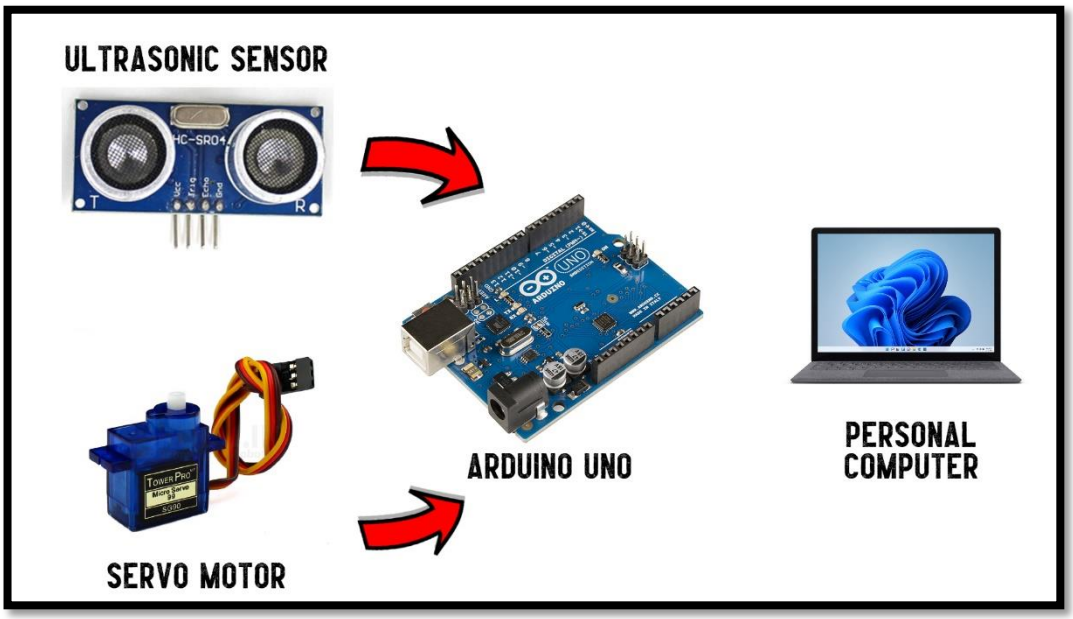
Target Detection and Tracking: By analysing the processed signals, the radar system can determine the presence, location, distance, velocity, and other properties of the detected objects.

Display and Visualization: The information obtained from the radar system's signal processing is typically displayed and visualized in a user-friendly format, such as a radar display or graphical user interface (GUI).

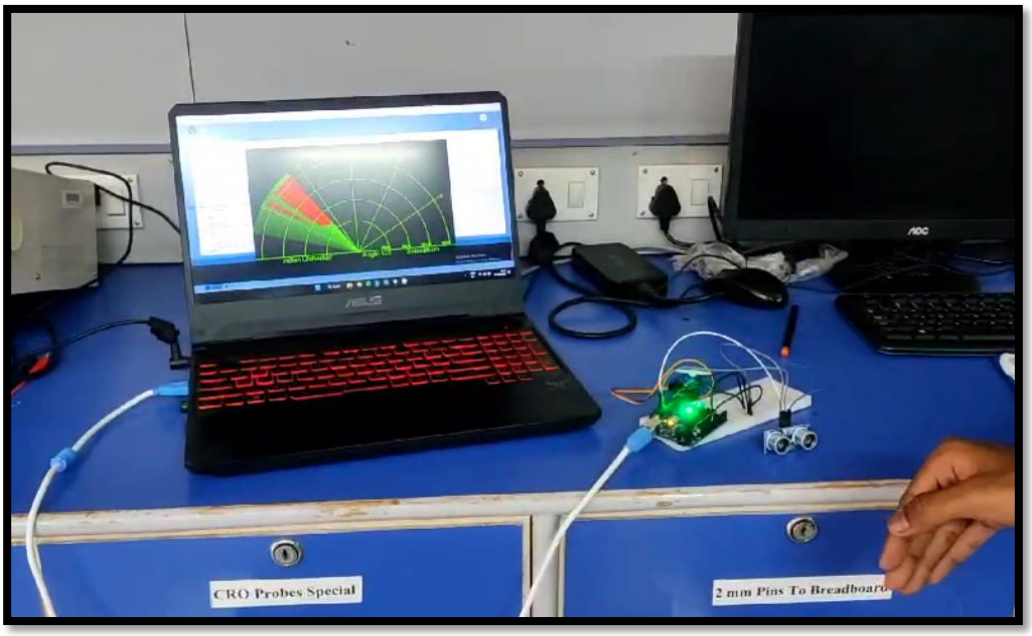
CIRCUIT DESCRIPTION



(Figure 1.1) IMPLEMENTATION OF CIRCUIT ON BREADBOARD



(Figure 1.2) BLOCK DIAGRAM OF THE SYSTEM



(Figure 1.3) ACTUAL IMAGE OF THE PROJECT

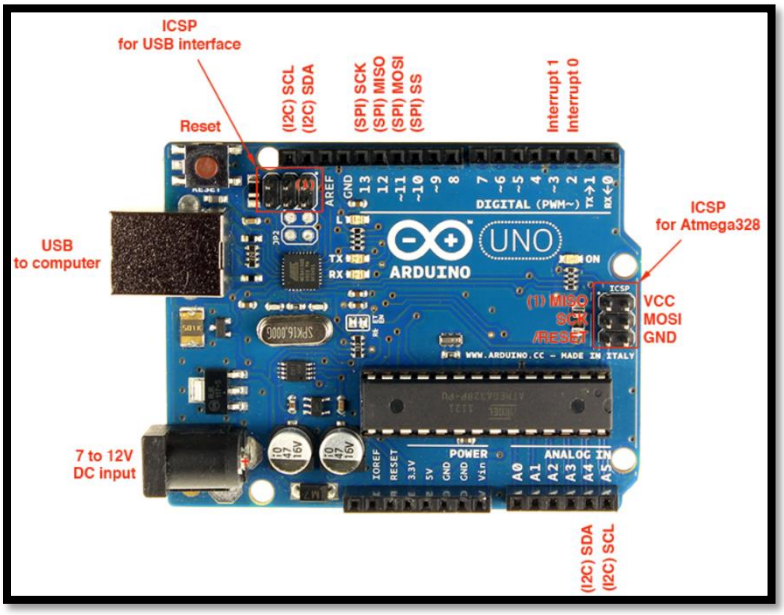
COMPONENT DESCRIPTION

1. ARDUINO UNO:

Arduino is based on microcontroller: **ATMEGA328P**. The analog ports A0-A5 accepts analog input. The analog input range is (0V to 5V) DC. Each channel has 10-bit ADC: 0 to 1023. A4 and A5 used for TWI (two wire interface). Upper 2 empty pins also used for TWI. TWI means we can interface multiple slaves and multiple masters (multiple Arduino boards).

The digital I/O pins D0-D13 accepts and provides digital I/O. D0 & D1 are not used for digital I/O rather used for serial I/O. Serial communication means exchanging data to-and-fro with other serial devices such as a PC. D3, D5, D6, D9, D10, D11 supports PWM output. D2 - D13 are used for digital I/O communication. D10, D11, D12, D13 used for SPI (serial peripheral interface) protocol. D0 and D1 used for TTL communication (communication between 2 digital devices).

Test led connected to D13 used for testing purposes. Rx - blinks when Arduino receives. Tx - blinks when Arduino transmits. The Arduino board works on a power supply of 7V to 12V. The user can take input voltage from 2 points on the board. USB-B port also used for connecting the board to a PC. Power supply. AREF (Analog Reference) pin used as an external input if both inputs are not working. Advantage of using an external reference is that in some cases we a get a less noisy signal. Additionally, we can directly provide input to (Vin) power pin.

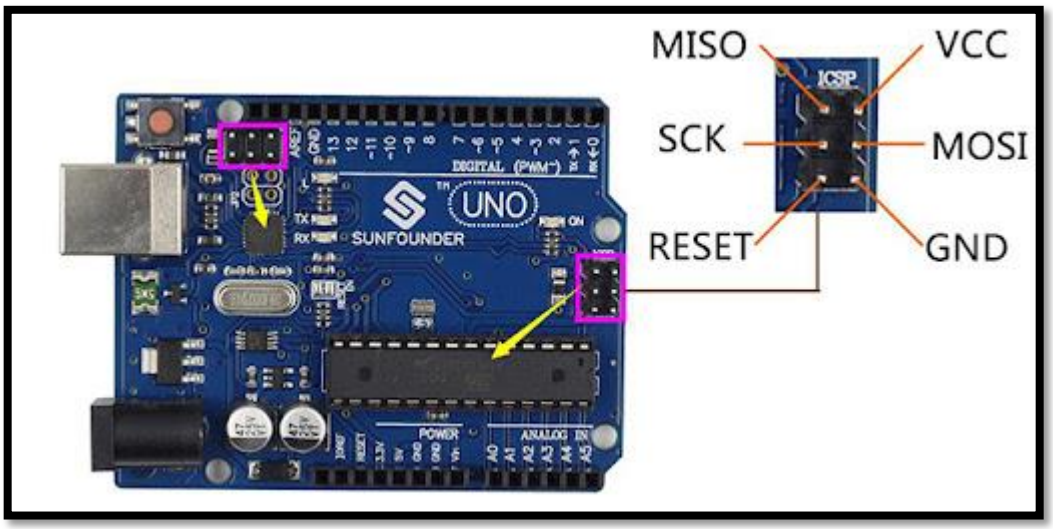


(Figure 1.4) ARDUINO UNO PIN OUT

Crytal oscillator = 16Mhz (grey oval) used to control speed of microcontroller. The "Y16.000" marking indicates that the device oscillates at 16 MHz. Interestingly, this oscillator regulates the device's ATmega16 microcontroller—which performs USB-to-serial conversion when interfacing with a computer—not the ATmega328P microcontroller. ATmega 16u2 chip: USB to serial conversion (Atmega 328 does not understand input).

Arduino board additionally consists of a voltage regulator: 1117C converts any input to 5V. 47uF capacitors (2): 1 connected to pin 1 of voltage regulator another connected to pin 3 of voltage regulator. A reverse current protection diode. Comparator IC (only used when USB and Power both supplied) compares both the voltages and passes 1 final voltage. 3.3V voltage regulator with 2 small capacitors.

ICSP for ATMEGA328P supports SPI (serial peripheral interface) used to upload bootloader since ATMEGA 328P when manufactured does not perform until bootloader is uploaded. ATMEGA328P consist of 32KB flash memory 0.5KB used by bootloader. ICSP for ATmega 16u2 chip. These pins are used to code and boot an Arduino from an external source. These pins allow inter-workings of two or more Arduino boards.



(Figure 1.5) ICSP HEADER PIN OUT

2. ULTRASONIC SENSOR HC-SR04:

The ultrasonic sensor emits ultrasound at 40,000 Hz which passes through the air, while it bounces back into the module if there is an object or obstacle in its way. The distance can be determined based on the travel time and the speed of the sound. The speed of the sound is around 0.34 centimetres per microsecond in air. This information and the time difference between transmitting and receiving the sound pulse are used by the Ultrasonic Sensor to calculate the distance from an object. The mathematical equation used is the following:

Distance =Time x Speed of Sound / 2

Time = time between when an ultrasonic wave is transmitted and when it is received. You divide this number by 2 because the sound wave must travel to the object and back. Transmitter generates ultrasonic waves in 8 burst of cycle at 40 KHz frequency.

Distance = Speed x Time

Distance = 0.34 x T cm

2d = 0.34 x T cm

$d = \frac{0.34}{2}$ x T cm

The configuration pin of HC-SR04 is VCC (1), TRIG (2), ECHO (3), and GND (4). The supply voltage of VCC is 5V and you attach TRIG and ECHO pin to any Digital I/O in your Arduino Board to power it.

(Figure 1.6) HC-S04 ULTRASONIC SENSOR

Specifications:

- Power Supply: DC 5V
- Working Current: 15mA
- Working Frequency: 40Hz
- Ranging Distance: 2cm – 400cm/4m
- Resolution: 0.3 cm
- Measuring Angle: 15 degrees
- Trigger Input Pulse width: 10uS
- Dimension: 45mm x 20mm x 15mm

4. SERVO MOTOR:

This servo motor is designed to rotate approximately 180 degrees, with 90 degrees in each direction, offering versatility in its range of motion. It can be easily monitored and controlled using any servo code, hardware, or library, providing flexibility for integration into different projects and systems.

For our project the servo motor rotates within angle range 15 degree to 165 degree and with a delay of 30 milliseconds.

Weight: The servo motor weighs only 9 grams, making it incredibly lightweight and suitable for applications where weight is a concern.

Dimension: It has compact dimensions of approximately 22.2 x 11.8 x 31 mm, making it small enough to fit into tight spaces or lightweight projects.

Stall torque: With a stall torque of 1.8 kgf cm, this servo motor provides high torque output relative to its size, enabling it to perform tasks that require significant force.

Operating speed: The servo motor operates at a rapid speed of 0.1 seconds per 60 degrees, allowing for quick and precise movements in applications such as robotics or remote-control systems.

Operating voltage: It operates within a voltage range of 4.8 V to approximately 5V, making it compatible with standard power sources commonly used in electronics projects.

Temperature range: The servo motor can function effectively within a temperature range of 0°C to 55°C, ensuring reliable performance in various environmental conditions.

(Figure 1.7) SERVO MOTOR

SOFTWARE DESCRIPTION

ARDUINO CODE:

```
#include <Servo.h>.
const int trigPin = 10;
const int echoPin = 11;
long duration;
int distance;
Servo myServo;
void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  Serial.begin(9600);
  myServo.attach(12);
}
void loop() {
  for(int i=15;i<=165;i++){
  myServo.write(i);
  delay(30);
  distance = calculateDistance();
  Serial.print(i);
  Serial.print(",");
  Serial.print(distance);
  Serial.print(".");
  }
  for(int i=165;i>15;i--){
  myServo.write(i);
  delay(30);
  distance = calculateDistance();
  Serial.print(i);
  Serial.print(",");
  Serial.print(distance);
  Serial.print(".");
  }
}
int calculateDistance(){
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH);
  distance= duration*0.034/2;
  return distance;
}
```

BREAKDOWN OF THE ABOVE CODE:

#include <Servo.h>

We first include the Servo library that allows control of servo motors.

Followed by defining the pins connected to the ultrasonic sensor (trigPin for transmitting signals and echoPin for receiving signals).

long duration; int distance;

These variables will be used to store the duration of the ultrasonic pulse and the calculated distance.

setup function

Sets the trigPin as an output and echoPin as an input.

Initializes serial communication for debugging (baud rate of 9600).

Attaches the servo motor to pin 12.

loop function

The loop function first sweeps the servo motor from 15 to 165 degrees, taking distance measurements at each angle.

Then it sweeps back from 165 to 15 degrees, again taking distance measurements.

For each position (i degrees), it calls calculateDistance() to get the distance measured by the ultrasonic sensor.

calculateDistance function

This function triggers the ultrasonic sensor by sending a pulse.

It then measures the duration for the pulse to return (using pulseIn function).

Calculates the distance based on the duration (assuming the speed of sound is 0.034 cm/microsecond).

Returns the calculated distance.

The delay() function in Arduino pauses the execution of the program for the specified number of milliseconds.

The duration of 30 milliseconds {delay(30)} might be adjusted based on the specific requirements and characteristics of your servo motor and application. It's used to provide enough time for the servo to physically move to the desired angle and settle down, ensuring accurate distance measurements can be taken without interference from servo motion.

PROCESSING CODE:

```
import processing.serial.*;
import java.awt.event.KeyEvent;
import java.io.IOException;
Serial myPort;
String angle="";
String distance="";
String data="";
String noObject;
float pixsDistance;
int iAngle, iDistance;
int index1=0;
int index2=0;
PFont orcFont;
void setup() {
 size (1200, 700);
 smooth();
 myPort = new Serial(this,"COM5", 9600);
 myPort.bufferUntil('.');
}
void draw() {
  fill(98,245,31);
  noStroke();
  fill(0,4);
  rect(0, 0, width, height-height*0.065);

  fill(98,245,31);
  drawRadar();
  drawLine();
  drawObject();
  drawText();
}
```

```
void serialEvent (Serial myPort) {
  data = myPort.readStringUntil('.');
  data = data.substring(0,data.length()-1);

  index1 = data.indexOf(",");
  angle= data.substring(0, index1);
  distance= data.substring(index1+1, data.length());
  iAngle = int(angle);
  iDistance = int(distance);
}
void drawRadar() {
  pushMatrix();
  translate(width/2,height-height*0.074);
  noFill();
  strokeWeight(2);
  stroke(98,245,31);
  arc(0,0,(width-width*0.0625),(width-width*0.0625),PI,TWO_PI);
  arc(0,0,(width-width*0.27),(width-width*0.27),PI,TWO_PI);
  arc(0,0,(width-width*0.479),(width-width*0.479),PI,TWO_PI);
  arc(0,0,(width-width*0.687),(width-width*0.687),PI,TWO_PI);
  line(-width/2,0,width/2,0);
  line(0,0,(-width/2)*cos(radians(30)),(-width/2)*sin(radians(30)));
  line(0,0,(-width/2)*cos(radians(60)),(-width/2)*sin(radians(60)));
  line(0,0,(-width/2)*cos(radians(90)),(-width/2)*sin(radians(90)));
  line(0,0,(-width/2)*cos(radians(120)),(-width/2)*sin(radians(120)));
  line(0,0,(-width/2)*cos(radians(150)),(-width/2)*sin(radians(150)));
  line((-width/2)*cos(radians(30)),0,width/2,0);
  popMatrix();
}
```

```
void drawObject() {
  pushMatrix();
  translate(width/2,height-height*0.074);
  strokeWeight(9);
  stroke(255,10,10);
  pixsDistance = iDistance*((height-height*0.1666)*0.025);
  if(iDistance<40){
  line(pixsDistance*cos(radians(iAngle)),-pixsDistance*sin(radians(iAngle)),(width-width*0.505)*cos(radians(iAngle
      )),-(width-width*0.505)*sin(radians(iAngle)));
  }
  popMatrix();
}
```

```
void drawLine() {
  pushMatrix();
  strokeWeight(9);
  stroke(30,250,60);
  translate(width/2,height-height*0.074);
  line(0,0,(height-height*0.12)*cos(radians(iAngle)),-(height-height*0.12)*sin(radians(iAngle)));
  popMatrix();
}
```

```
void drawText() {
  pushMatrix();
  if(iDistance>40) {
  noObject = "Out of Range"; }
  else { noObject = "In Range"; }
  fill(0,0,0);
  noStroke();
  rect(0, height-height*0.0648, width, height);
  fill(98,245,31);
  textSize(25);
  text("10cm",width-width*0.3854,height-height*0.0833);
  text("20cm",width-width*0.281,height-height*0.0833);
  text("30cm",width-width*0.177,height-height*0.0833);
  text("40cm",width-width*0.0729,height-height*0.0833);
  textSize(40);
  text("SEM 4", width-width*0.875, height-height*0.0277);
  text("Angle: " + iAngle +"  ", width-width*0.48, height-height*0.0277);
  text("Distance: ", width-width*0.26, height-height*0.0277);
```

```
  if(iDistance<40) {
  text("        " + iDistance +" cm", width-width*0.225, height-height*0.0277); }
  textSize(25);
  fill(98,245,60);
  translate((width-width*0.4994)+width/2*cos(radians(30)),(height-height*0.0907)-width/2*sin(radians(30
      )));
  rotate(-radians(-60));
  text("30 ",0,0);
  resetMatrix();
  translate((width-width*0.503)+width/2*cos(radians(60)),(height-height*0.0888)-width/2*sin(radians(60)));
  rotate(-radians(-30));
  text("60 ",0,0);
  resetMatrix();
  translate((width-width*0.507)+width/2*cos(radians(90)),(height-height*0.0833)-width/2*sin(radians(90)));
  rotate(radians(0));
  text("90 ",0,0);
  resetMatrix();
  translate(width-width*0.513+width/2*cos(radians(120)),(height-height*0.07129)-width/2*sin(radians(120
      )));
  rotate(radians(-30));
  text("120 ",0,0);
  resetMatrix();
  translate((width-width*0.5104)+width/2*cos(radians(150)),(height-height*0.0574)-width/2*sin(radians(150
      )));
  rotate(radians(-60));
  text("150 ",0,0);
  popMatrix();
}
```

ADVANTAGES

1. Affordability and Accessibility: Arduino boards, servo motors, and ultrasonic sensors are relatively affordable and widely available, making them accessible for hobbyists and educational projects.

2. Easy Integration: Arduino's open-source platform simplifies the integration of hardware components like servo motors and ultrasonic sensors, enabling rapid prototyping and development.

3. Precise Angle Control: Servo motors provide precise angular control, allowing the radar to scan specific angles accurately for target detection.

4. Simple Interface: Arduino's programming environment and libraries make it straightforward to control servo motors and read sensor data, reducing development complexity.

5. Real-Time Data Acquisition: Ultrasonic sensors provide real-time distance measurements, allowing dynamic adjustments in radar scanning based on detected distances.

6. Compact and Portable: Arduino-based radar systems are compact and portable, suitable for small-scale applications and demonstrations.

7. Customizable and Expandable: Arduino projects can be easily customized and expanded by incorporating additional sensors or functionalities for enhanced radar performance.

DISADVANTAGES

1. Limited Range: Ultrasonic sensors have a limited detection range (typically up to a few meters), restricting the effective range of the radar system.

2. Inaccuracies in Measurement: Ultrasonic sensors may suffer from inaccuracies due to environmental factors like temperature, humidity, and surface material.

3. Mechanical Noise: Servo motors can generate mechanical noise during operation, potentially affecting sensor readings and overall system performance.

4. Complex Calibration: Achieving optimal performance may require calibration and fine-tuning of sensor parameters, which can be time-consuming.

5. Interference and False Readings: External noise or interference can sometimes lead to false distance readings, affecting the reliability of the radar system.

6. Limited Angular Range: Servo motors have a limited angular range (e.g., 0 to 180 degrees), which may constrain the field of view for radar scanning.

SCOPE

1. Education and Learning: Ideal for educational purposes to understand basic radar principles, sensor interfacing, and microcontroller programming.

2. DIY Projects: Suitable for DIY enthusiasts and hobbyists interested in building simple radar systems for personal use or experimentation.

3. Prototyping: Enables rapid prototyping of short-range radar concepts and functionalities for proof-of-concept demonstrations.

4. Monitoring and Surveillance: Can be used for basic monitoring and surveillance applications in controlled environments.

5. Automation and Robotics: Integration into robotics projects for obstacle detection and avoidance applications.

CONCLUSION

Using Arduino with servo motors and ultrasonic sensors for short-range radar detection offers a cost-effective and accessible solution for prototyping and educational purposes. While these components provide precise control and real-time distance measurements, they also have limitations such as restricted range, potential inaccuracies, and calibration requirements.

For hobbyists and beginners, this setup serves as an excellent platform to learn about sensor integration, data acquisition, and control systems. However, for more sophisticated radar applications requiring longer ranges and higher precision, alternative sensor technologies and hardware platforms may be more suitable. Overall, Arduino-based radar systems using servo motors and ultrasonic sensors offer a valuable entry point into radar technology, fostering creativity and innovation in the field of electronics and robotics.

REFERENCES

1. **Reddit r/arduino** - reddit.com/r/arduino
2. **Beginning Arduino** by Michael McRoberts
3. **Electronics Hub** - electronicshub.org
4. **Arduino Forum** - forum.arduino.cc